케이녹 C언어 6회차 과제

17기 최동현 (202511688)

1. 동적 할당 (Dynamic Allocation)

• 동적 할당(Dynamic Memory Allocation)이란?

: C언어에서 **동적할당**은 **스택(Stack)**과는 다르게 **프로그램 실행(Runtime) 중**에 **필요한 만큼**의 메모리를 운영체제로부터 **힙(Heap) 영역**에 **할당** 받고 **반납**하는 메모리 관리 방식입니다.

스택과 달리 **크기 제한**이 **유연**하다는 장점이 있지만, 스택보다 **느리며** 개발자가 직접 메모리 **생명 주기(Life Cycle)**를 완벽하게 제어해야 합니다. C언어 표준 라이브러리 함수로 **malloc()**, **calloc()**, **realloc()** 들이 존재합니다.

```c
#include <stdlib.h> // 동적할당 사용시 포함

// malloc(Memory Allocation) 할당 형식
자료형* 포인터명 = (자료형*)malloc(할당할_바이트_크기);

// calloc(Continuous Allocation) 할당 형식
자료형* 포인터명 = (자료형*)calloc(요소_개수, 요소_단일_크기);

// realloc(Re-Allocation) 재할당 형식
포인터명 = (자료형*)realloc(기존_포인터명, 새로운_바이트_크기);

// 메모리 해제 형식
free(포인터명);
```

1-1. 동적 할당 (Dynamic Allocation)

• 동적 할당(Dynamic Memory Allocation)을 사용하는 이유?

: C언어에서 **스택(Stack)** 말고 굳이 이런 **동적 할당**을 **사용하는 이유**는 **스택의 한계** 때문에 있습니다.
스택이 가지고 있지 않은 **동적 할당**의 **장점**은 **크게 두가지**가 존재 합니다. (**+ 스택**은 ***1MB ~ 8MB 한계***)

1) 얼마나 필요한지 미리 **알 수 없을 때** (*가변 크기*)

- C언어에서 배열 선언(스택)은 **상수**로만 **크기**를 정해야 함. (데이터 100개를 입력할지, 1000개를 모를 때 문제 발생)
- example) scanf("%d", &index);
- int* arr = (int*)malloc(sizeof(int) * index); // 사용자가 입력한 값에 따라 배열 크기가 결정 됨.

2) 함수가 끝나도 데이터를 **유지**해야 할 때 (*수명 연장*)

- **함수 내부**에서 **생성**한 데이터를 **호출한 곳(caller)**에서 넘겨주어 계속 **사용**해야 할 때, 유용합니다.
- example)
- int* createData() {

```
int* data = (int*)malloc(sizeof(int));
*data = 10;
return data; // 힙(Heap)에 할당된 주소를 반환하므로 함수가 끝나도 유지 함.
}
```

1-2. 동적 할당 (Dynamic Allocation)

- **스택(Stack) vs 힙(Heap) 비교**

  : 결론적으로 언제 뭘 선택해야 하나?

	스택 (Stack)	힙 (Heap)
할당 방식	정적/자동 할당	개발자 직접 할당
해제 시점	함수 종료 시 자동 해제	free() 호출 전 까지 유지 (수동)
속도	힙 보다 매우 빠름	스택 보다 상대적으로 느림
한계	크기 작고, 크기 고정 됨 (Stack Overflow의 위험)	원하면 메모리 전체 만큼 크게 사용 가능

1-3. 동적 할당 (Dynamic Allocation)

- **malloc(M : Memory), calloc(C : Continuous), realloc(R : Resize), free**

  : **malloc(size);** 방식으로 할당하며, 지정된 **바이트 크기만큼** 힙 메모리 영역에 할당합니다. [주의: 내부 데이터 *초기화X, 쓰레기 값(Garbage Value)* 그대로 가짐]

  calloc(count, size); 방식으로 할당하며, **지정된 데이터 개수**와 **크기만큼 할당**합니다. (모든 메모리 비트 **0**으로 초기화)

  realloc(ptr, size); 방식으로 할당하며, 이미 할당 된 메모리 공간의 **크기**를 **확장**하거나 **축소**하는 기능을 합니다. (뒤 메모리 **공간**이 **부족**할 시 새로운 메모리 영역을 찾아 할당. 이 때 **메모리 주소**가 **달라질** 수 있음)

  free(ptr); 로 할당된 힙 메모리 공간을 운영체제(Heap Allocator)에 **반납**합니다.

1-4. 동적 할당 (Dynamic Allocation)

- C언어 동적 할당 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *arr = (int*)malloc(size: sizeof(int) * 5); // int(4) * 5
    arr[0] = 10; // 0번 방에 10 할당

    arr = (int*)realloc(ptr: arr, size: sizeof(int) * 10); // int(4) * 10로 재할당
    arr[9] = 100; // 9번 방에 100할당

    printf("arr[0]: %d\narr[9]: %d\n", *(arr+0), *(arr+9));
    free(arr); // 반납 필수!
    arr = NULL; // UAF(Use-After-Free) 방지

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day6/main.c

arr[0]: 10
arr[9]: 100

계속하려면 엔터 혹은 명령을 입력하십시오
```

1-5. 동적 할당 (주의 사항!)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char* adminPassword = (char*)malloc(size: 16);
    strcpy(adminPassword, "K-knock-Pwnable");

    printf("adminPassword address: %p\n", (void*)adminPassword);
    free(adminPassword);

    char* userBuffer = (char*)malloc(size: 16);
    printf("userBuffer address: %p\n", (void*)userBuffer);

    for (int i = 0; i < 16; i++) { // Safe-Linking 때문에 반복문 돌림
        unsigned char byteValue = (unsigned char)userBuffer[i];
        // 아스키는 음수가 없어서 unsigned
        char charValue = (byteValue >= 32 && byteValue <= 126) ? byteValue : '0';
        // 아스키 Space부터 Del까지
        printf("INDEX %2d: (%c)[ 0x%02X ]\n", i, charValue, byteValue);
    }
    free(userBuffer);

    return 0;
}
```

1-5. 동적 할당 (주의 사항!)

```
root@localhost:~# ./vuln
adminPassword address: 0x1d1f1a0
userBuffer address: 0x1d1f1a0
INDEX  0: (0)[ 0x1F ]
INDEX  1: (0)[ 0x1D ]
INDEX  2: (0)[ 0x00 ]
INDEX  3: (0)[ 0x00 ]
INDEX  4: (0)[ 0x00 ]
INDEX  5: (0)[ 0x00 ]
INDEX  6: (0)[ 0x00 ]
INDEX  7: (0)[ 0x00 ]
INDEX  8: (P)[ 0x50 ]
INDEX  9: (w)[ 0x77 ]
INDEX 10: (n)[ 0x6E ]
INDEX 11: (a)[ 0x61 ]
INDEX 12: (b)[ 0x62 ]
INDEX 13: (l)[ 0x6C ]
INDEX 14: (e)[ 0x65 ]
INDEX 15: (0)[ 0x00 ]
```

두 배열 전부 같은 주소를 할당받음

0x1d1f1a0 -> 0x1d1f
(우측으로 12비트 쉬프트)

0x0000000000001d1f
(64bit 포인터 최대 자리수)
후에 Pwnable 출력

리눅스 GCC 환경

☆ **malloc 할당** 시 **calloc**과 다르게 메모리 비트를 0x00으로 초기화하지 않고 기존에 남아있던 쓰레기 값(Garbage Value)이 남아있기에, 전에 할당했던 ***"K-knock-Pwnable"*** 중 Safe-Linking에 의해 ***"Pwnable"***이 출력 되는 현상을 볼 수 있습니다.

☆ 이런 현상 및 취약점을 **UAF(Use-After-Free)**라고 합니다. 때문에 ***ptr = NULL;*** 을 하는 등의 **주의 필요!**

1-6. 동적 할당 (Dynamic Allocation)

- **메모리 누수(Memory Leak), 댕글링 포인터(Dangling Pointer)란?**

:

1) 메모리 누수(Memory Leak)는 **malloc**으로 **할당**받은 메모리를 **free**로 **반납하지 않아**, 프로그램이 **점유**하는 **메모리**가 지속적으로 **증가**하는 **결함**입니다. 메모리 누수로 **힙(Heap)** 영역을 넘어가면 서비스 거부(Denial of Service) 상태를 유발합니다.

2) 댕글링 포인터(Dangling Pointer)는 **free**로 메모리 **반납**을 했음에도 불구하고 **포인터 변수** 자체는 여전히 **해제된 메모리**의 **주소 값**이 **남아 있는 상태**를 의미 합니다. (허공을 가리키는 포인터)

```
// 메모리 누수(Memory Leak) 방지
free(ptr); // 이때 포인터는 댕글링 포인터가 됨

// 댕글링 포인터(Dangling Pointer) 방지
ptr = NULL; // free로 해제 후 포인터를 초기화 하여 접근을 원천차단
```

2-1. 구조체 (Structure Syntax)

• 구조체(Structure Syntax)란?

: C언어에서 **구조체(Structure Syntax)**는 **여러 개**의 **서로 다른 자료형**을 **묶어**서 하나의 **새로운 데이터 단위(Type)**를 **정의**하는 **문법**입니다. (메모리 상에 여러 데이터가 인접하게 배치됨)

그 중 **구조체 배열(Array of Structure)**은 **구조체로 정의된 데이터 집합**을 **여러 개 연속**으로 **배치**하는 방식입니다. 메모리 상에는 구조체의 모든 멤버가 **선형**적으로 쭉 나열됩니다. (ex. Member1, Member2... 순서로 메모리에 할당 됨)

```
// 구조체(Structure Syntax) 정의
struct 구조체이름 {
    자료형 멤버1;
    자료형 멤버2;
};

// 구조체 배열(Array of Structure) 선언 및 초기화
struct 구조체이름 배열명[배열크기] = {
    {초기값1, 초기값2},
    {초기값3, 초기값4}
};
```

2-1. 구조체 (Structure Syntax)

- C언어 구조체 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>
#include <stdlib.h>

struct person {
    int num;
    char name[20];
};

int main(void) {
    struct person list[2] = {
        [0]={.num=1, .name="Donghyun"},
        [1]={.num=2, .name="Inhyuk"}
    };

    for(int i = 0; i < 2; i++) {
        printf("[person%d의 정보]\n번호: %d, 이름: %s\n",
                i, list[i].num, list[i].name);
    }

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day6/main.c

[person0의 정보]
번호: 1, 이름: Donghyun
[person1의 정보]
번호: 2, 이름: Inhyuk

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-2. 구조체 (주의 사항!)

```c
#include <stdio.h>
#include <string.h>

struct session {
    char username[8]; // 배열 크기 1Byte * 8임
    int isAdmin; // 스택 정렬해도 바로 다음 메모리가 isAdmin임
};

int main(void) {
    struct session list[2];
    list[0].isAdmin = 0;
    list[1].isAdmin = 0;

    for(int i = 0; i < 2; i++) {
        printf("사용자 이름을 입력하세요: ");
        gets(list[0].username); // 버퍼 크기제한 없는 gets를 사용함 (취약)
    }
    if (list[0].isAdmin != 0 || list[1].isAdmin != 0) {
        printf("관리자 권한 획득 성공!\n");
    }
    return 0;
}
```

```
1 warning generated.
warning: this program uses gets(), which is unsafe.
사용자 이름을 입력하세요: test
사용자 이름을 입력하세요: AAAAAAAA1
관리자 권한 획득 성공!

계속하려면 엔터 혹은 명령을 입력하십시오
```

배열 크기 8을 넘어감

2-3. 구조체 (Structure Syntax)

• 구조체 포인터(Pointer to a Structure)란?

: C언어에서 **구조체 포인터(Struct Pointer)**는 **구조체가 할당**된 메모리의 **시작 주소**를 가리키는 **포인터**입니다.
구조체는 바이트가 크기 때문에 ***Call by Value*** 보다는 ***Call by Reference*** 형식을 거의 필수적으로 사용합니다.
(+ **포인터**를 통해 **구조체 멤버**에 **접근** 할 때는 [.] 대신 [->]를 사용합니다. ***ptr->member1***)

```
// 구조체 변수 선언
struct 구조체이름 구조체변수명 = {초기값들};

// 구조체 포인터(Struct Pointer) 선언 및 초기화
struct 구조체이름 *포인터명 = &구조체변수명;

// 포인터(Pointer)를 통한 구조체(Struct) 멤버 접근 방식
포인터명->멤버이름 = 변경할값;
```

2-3. 구조체 (Structure Syntax)

- C언어 구조체 포인터 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>
#include <stdlib.h>

struct STRT {
    int num;
};

void modify(struct STRT *ptr) {
    ptr->num = 100; // 정수 100
}

int main(void) {
    struct STRT var = {.num=10}; // 정수 10

    modify(ptr: &var); // Call by reference
    printf("구조체 변수 num의 값 : %d\n", var.num);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day6/main.c

구조체 변수 num의 값 : 100

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-4. 구조체 (Structure Syntax)

- **공용체(Union)와 열거형(Enum)이란?**
  1) **공용체(Union):** 구조체와 **형태**는 **같지만**, 다른 점으로 **메모리 할당**이 **다른 방식**입니다. 내부에 정의된 모든 멤버가 **동일한 메모리 시작 주소**를 **공유**합니다. 공용체의 전체 크기는 멤버 중 **가장 큰 자료형**의 **크기**로 결정됩니다. 한 멤버에 값을 쓰면 다른 멤버의 값도 **물리적으로 동일한 비트**를 **공유**하기에 값이 변형됩니다.
  2) **열거형(Enum): 정수형 상수**에 **이름**을 **부여**하여 **그룹화**하는 문법입니다. 내부적으로 0부터 시작하는 정수로 처리됩니다. (ex. Monday = 0, Tuesday = 1, …)

```
// 공용체(Union) 선언 및 초기화
union 공용체이름 {
    자료형 멤버1;
    자료형 멤버2;
};
union 공용체이름 공용체변수명;
공용체변수명.멤버 = 초기화;

// 열거형(Enum) 선언 및 초기화
enum 열거형이름 { 상수명1 = 값, 상수명2 = 값 };
enum 열거형이름 열거형변수명 = 상수명;
```

2-5. 구조체 (Structure Syntax)

- C언어 공용체 및 열거형 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

union shareMemory {
    long long intValue; // 8Byte
    char charValue[7]; // 1 * 7 = 7Byte
    // long long이나 char 둘 중
    // 더 큰 자료형(long long)으로 전체 크기 잡힘
};

enum state { FAIL = 0, SUCCESS = 1 };

int main(void) {
    union shareMemory sm;
    enum state st = SUCCESS;

    sm.intValue = 0x4B434F4E4B4B; // [4B : K], [4E : N], [4F : O], [43 : C]

    printf("[UNION] 정수 출력 : %lld\n", sm.intValue);
    printf("[UNION] 문자열 출력 : %s\n", sm.charValue);
    printf("[ENUM] 상태 : %d\n", st);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day6/main.c

[UNION] 정수 출력 : 82752465423179
[UNION] 문자열 출력 : KKNOCK
[ENUM] 상태 : 1

계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 실습 과제 (문자열 대체 프로그램)

문자열 대체 프로그램 과제

목표

이 과제는 C 언어에서 포인터와 함수 호출 방식의 이해를 확인하기 위한 것입니다. 문자열 내에서 특정 문자열을 찾아 새로운 문자열로 대체하는 프로그램을 구현해야 합니다.

요구사항

1. 사용자로부터 세 가지 입력을 받으세요:
   - 원본 문자열
   - 대체할 문자열(찾을 문자열)
   - 새로운 문자열(바꿀 문자열)
2. 다음 기능을 가진 `replace` 함수를 구현하세요:

```c
/**
 * @brief 문자열에서 특정 문자열을 새로운 문자열로 대체하는 함수
 *
 * @param str 수정할 문자열의 포인터
 * @param old_rep 대체할 문자열
 * @param new_rep 새로운 문자열
 * @return int 대체된 문자열 개수
 */
int replace(char str[], char old_rep[], char new_rep[]);
```

예시 실행 결과

```
문자열을 입력하세요: Hello world, Hello everyone!
대체할 문자열을 입력하세요: Hello
새 문자열을 입력하세요: Hi
변경된 문자열: Hi world, Hi everyone!
대체된 문자열 개수: 2
```

3. 실습 과제 (문자열 대체 프로그램)

```c
int replace(char str[], char old_rep[], char new_rep[]) {
    char buf[100]; // 문자열에 수정해서 붙여 넣을 버퍼
    int count = 0; // 카운트 변수
    int strIdx = 0, bufIdx = 0; // 문자열, 버퍼 인덱스
    int oldRepLength = strlen(s: old_rep); // 바꿀 문자열 길이 계산
    int newRepLength = strlen(s: new_rep); // 바뀔 문자열 길이

    if (oldRepLength == 0) return 0; // 바꿀 문자열 길이가 0이면 그냥 끝

    while (str[strIdx] != '\0') { // 원본 문자열 끝날때까지 반복
        if (equal(str1: str + strIdx, str2: old_rep)) {
            // 일치 시작하는 곳에서 바꿀 문자열과 비교
            // 만약 같다면
            strcpy(buf + bufIdx, new_rep);
            // 바뀔 문자열을 버퍼에 넣고
            strIdx += oldRepLength;
            // 버퍼에 썼으니 문자열 인덱스에 길이 만큼 건너뛰기
            bufIdx += newRepLength;
            // 버퍼도 똑같이 인덱스 길이 건너뛰고
            count++; // 문자열 개수 카운트하기
        } else { // 일치하지 않는 다면
            buf[bufIdx] = str[strIdx]; // 일단 일치부분은 버퍼에 씀
            strIdx++; // 그리고 인덱스 기록
            bufIdx++; // 버퍼 인덱스 또한 길이 이동
        }
    }
    buf[bufIdx] = '\0';
    // 이거 안하면 문자열 아니어서 뒤 쓰레기값 땜에 깨짐. 버퍼 끝에 널추가
    strcpy(str, buf); // 수정한 문자열을 진짜 문자열에 복사
    return count; // 수정된 문자열 개수 반환
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day5/replace.c

문자열을 입력하세요: Hello world, Hello everyone!
대체할 문자열을 입력하세요: Hello
새 문자열을 입력하세요: Hi
변경된 문자열: Hi world, Hi everyone!
대체된 문자열 개수: 2

계속하려면 엔터 혹은 명령을 입력하십시오
```

감사합니다

17기 최동현